

Rocky Project Report

Eliam Batista, Kaman Chen, Mehmet Firat

Identification of Motor Constants

To determine the constants of the Balance Robot, the motor decay rate α and the motor gain β , the motor behavior described by a first order system and use a transfer function $M(s)$ of velocity of the wheel $V(s)$ to the motor (velocity) control signal, $Y(s)$.

$$M(s) = \frac{V(s)}{Y(s)} = \frac{\alpha\beta}{s + \alpha} = \frac{K/\tau}{s + 1/\tau}$$

To identify these parameters with the robot we use, motor calibration data were collected using an Arduino microcontroller and CoolTerm. During this experiment, step commands were applied to the wheel motors while the wheel velocities were measured and recorded. The collected data is then imported into MATLAB for analysis and visualization.

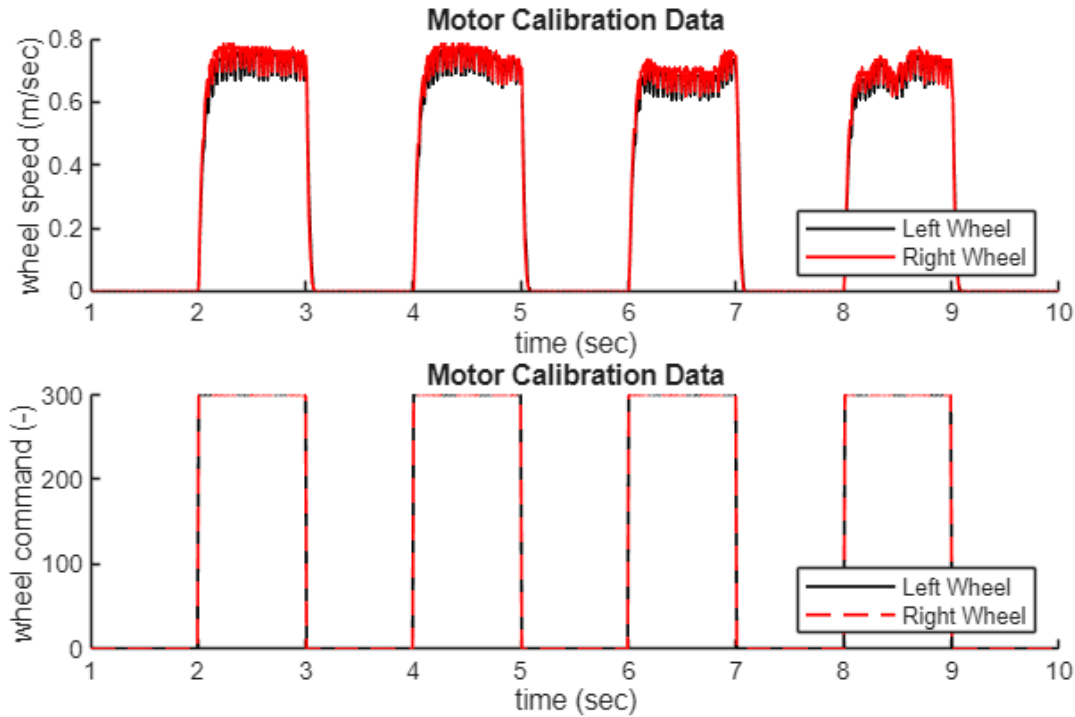


Figure 1: Motor calibration data showing the response of the left and right wheels to step wheel command inputs. The top plot shows the measured wheel speeds (m/s), and the bottom plot shows the applied wheel command signals over time.

The data of both left and right wheel's velocity and position while they receive the order command are visualized after the measurements. The bottom plot shows the wheel command signals that are applied to motors, with step inputs of approximately 300. The upper plot shows the measured wheel speeds for both the left and right wheels in response to the wheel commands. The left wheel and the right wheel are closely matched and they both respond to the command and reach their operating speed fast, indicating that both motors have pretty similar characteristics, such as the motor gain and their time constant is small.

To determine the specific value of the motor decay rate α and the motor gain β , the measured data were fitted to the first order response model by using MATLAB's curve fitting function

$$v(t) = c(1 - e^{-at})$$

Where C is the steady state velocity and a is the exponential decay rate of the response.

The prediction starts with initial guesses for parameters a and c for either left or right velocity. The wheel velocity's linear fit with the model and initial guess will adjust the parameter's values to minimize the error between the model prediction and the measured data.

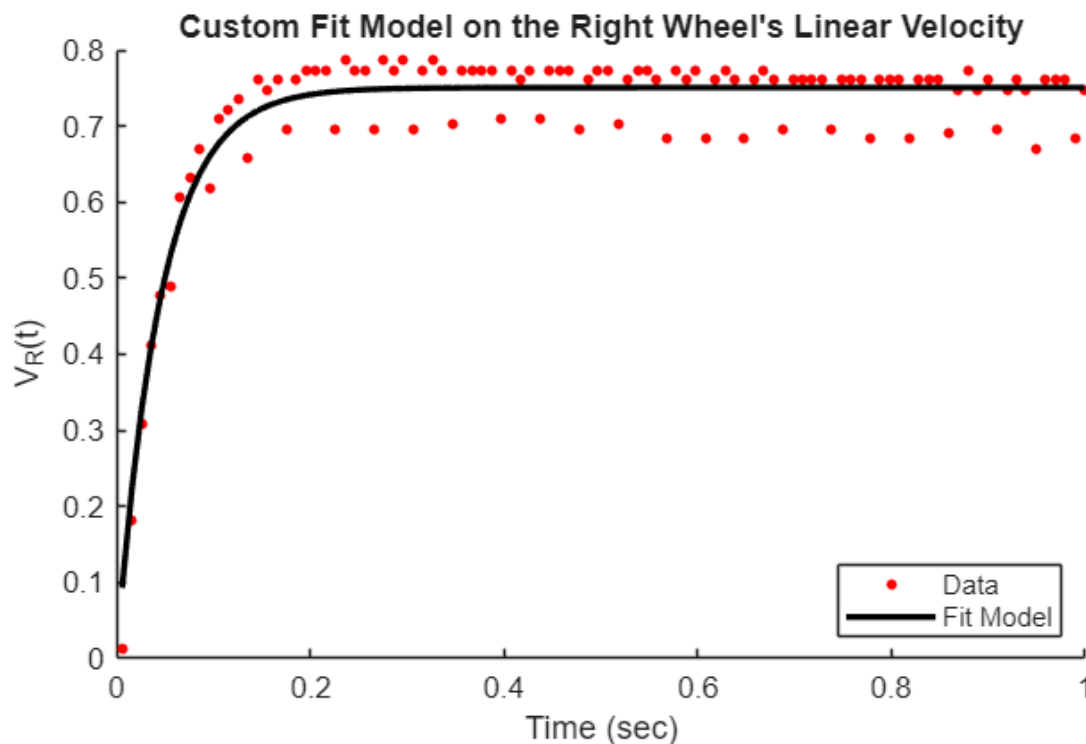


Figure 2: Comparison of right-wheel linear velocity response with experimental data (red dots) and the fitted first-order model (black line). It captures the rise transient and steady-state behavior of the wheel velocity.

The fitted model illustrates the state of the wheel when the motor command is applied: the wheel speed rapidly increases from zero and gradually approaches a steady-state value. And It matches with the trend that the measurement data show. From the result of the best fit function, the decay rate a was determined to be approximately **$a = 21.98$** .

As we know $\alpha = \frac{1}{\tau}$, The motor time constant is **$\tau = 0.0455$** . The gain β calculated from the steady state velocity divided by the magnitude of the motor command

$$b = \frac{c}{Y}, \quad \beta = 0.0025.$$

Identification of Natural Frequency and Effective Pendulum Length

To determine the natural frequency ω_n and effective length, l_{eff} of the robot's pendulum body, the motion of the robot is approximated by using a second-order differential equation.

$$\ddot{\theta} + 2\zeta\omega_n\dot{\theta} + \omega_n^2 = 0, \quad \omega_n^2 = \frac{g}{l_{eff}}$$

Where θ is the body angle, ζ is the damping ratio, and ω_n is the natural frequency. The data were collected by letting the robot to swing freely so its oscillatory response could be recorded by Arduino. Then the measured data of angular moments were processed with the model by using the `regress_underdamped_response` and visualized in MATLAB. The result of the fitting function determines that natural frequency **$\omega_n = 4.096$ rad/sec**.

To verify the accuracy of the model, the fitted parameters were substituted into the analytical solution of the underdamped oscillation equation, and compared with the measured data.

$$\theta(t) = D + e^{-\sigma t} (C \cos(\omega_d t) + S \sin(\omega_d t))$$

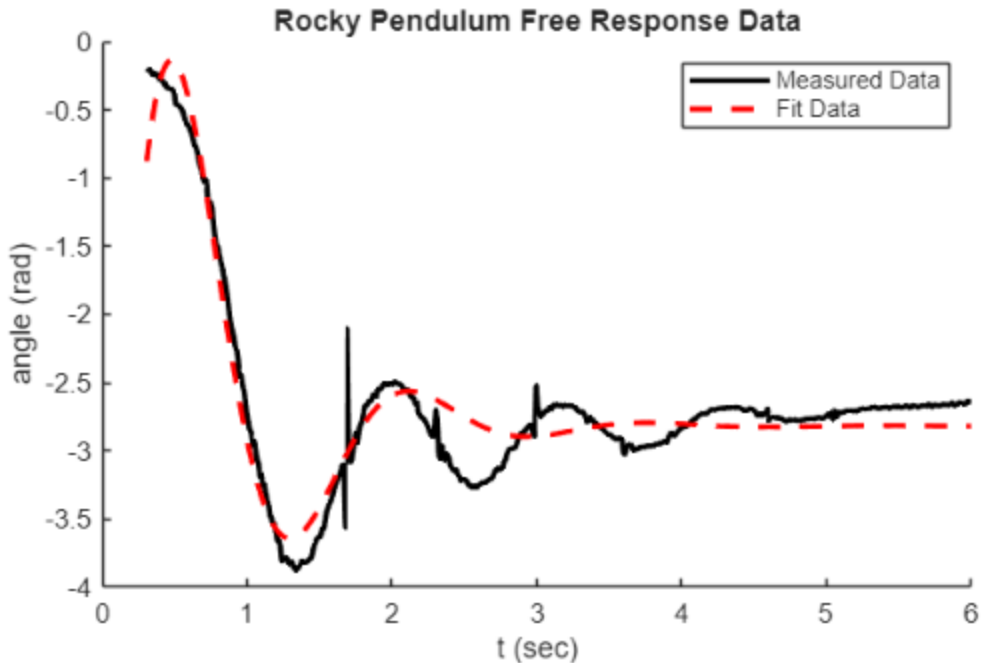


Figure 3: free response of a rocky pendulum showing measured angular displacement compared with a fitted model. The black solid line represents the experimental data, while the red dashed line shows the model fit. It shows a decaying oscillation as the system gradually approaches a steady state.

The plot of the pendulum free response shows the rocky pendulum system's behavior over time. The measured angular displacement (black line) initially starts near zero and rapidly decreases. It reaches a large negative displacement within the first second, then the amplitude of the oscillatory motion continues to decrease. It gradually weakened over time until it reached a steady state. The red dash line represents the model fit. Overall, It followed the trend of the measured data closely but didn't catch it perfectly by reaching the steady state faster and ignoring fluctuation caused by noises that happen during the measurement process. However, the fitted model accurately represents the overall behavior of the system. Minor differences are expected due to sensor noise and measurement imperfections.

With the known natural frequency, the effective pendulum length computed by the

$$\omega_n^2 = \frac{g}{l_{eff}}$$

formula, where g is gravity = 9.81 m/s^2 . So the $l_{eff} = g / \omega_n^2$, and by substituting the natural frequency ω_n , we get the value of $l_{eff} = 0.5846 \text{ m}$.

Initial System: Disturbance Response

The balancing robot system combines both motor and pendulum dynamics; Physically, the system behaves as an inverted pendulum mounted on a moving cart, which is naturally unstable. Any small disturbance would cause the body angle to increase and fall if there was no controller. The motor dynamics introduced by a first order system with the transfer function.

$$M(s) = \frac{V(s)}{Y(s)} = \frac{\alpha\beta}{s + \alpha} :$$

Where α is the decay rate and β is the gain, which relates the input motor command to the resulting wheel velocity. And the pendulum dynamics describe the relationship between the cart motion and the body as

$$H_{v\theta}(s) = \frac{\Theta(s)}{V(s)} = \frac{s/l_{eff}}{s^2 - \omega_n^2}$$

Where l_{eff} is effective pendulum length and ω_n is the natural frequency. By combining the transfer functions of motor and pendulum gives the open loop transfer function of the system.

$$\frac{\Theta(s)}{Y(s)} = \left(\frac{\alpha\beta}{s + \alpha} \right) \left(\frac{s/l_{eff}}{s^2 - \omega_n^2} \right)$$

The poles of the system are determined by the denominator, which is $(s+\alpha)(s^2-\omega_n^2)$, and show the pole of $s = -\alpha, \pm\omega_n$. The pole located at $s = +\omega_n$ lies in the right half of the complex plane, which shows that the system is unstable and needs the proportional integral (PI) controller to help stabilize the situation. The transfer function joins the

$$K(s) = \frac{K_p s + K_i}{s} = K_p + \frac{K_i}{s}$$

combination. . The system becomes a closed loop system after the PI controller joins. Instead of the natural unstable poles, we choose desired pole location to help produce the result we want.

$$\frac{\Theta(s)}{D(s)} = \frac{(s + \alpha)(s^2 - \omega_n^2)}{s^3 + \alpha s^2 + (K_p \alpha \beta / l_{eff} - \omega_n^2)s + (K_i \alpha \beta / l_{eff} - \omega_n^2 \alpha)}$$

This show use the pole relationship with

$$p_1 = -\frac{\alpha}{3}, \quad p_2 = -\frac{\alpha}{3} + iq, \quad p_3 = -\frac{\alpha}{3} - iq$$

Which means that the average location of the poles is limited to $-\alpha/3$ in the real axis. By looking for the desired pole, we choose the damping ratio ζ and natural frequency ω_n based on the performance specifications of the system. We selected 0.8 for the damping ratio because we expect to get stable response, quick settling and the imagery pole to be closer to the real axis. So a larger damping ratio like 0.8 would be a good number. And we choose $\pi/2$ as our natural frequency because it determines how fast the system responds and we expect faster reaction and correction.

Poles of a system in the complex plane are represented as points with real and imaginary parts. Geometrically, this corresponds to a right triangle where the hypotenuse is the natural frequency ω_n , and the angle θ between the negative real axis and the pole relates to the damping ratio by: $\cos(\theta) = \zeta$. The angle would be calculated as $\theta = \cos^{-1}(\zeta)$. Then the real part of the pole, which is $-\alpha/3$ calculated by equation : $-3 \cdot \omega_n \cdot \cos(\theta) = -3.7699$ and the imagery part q would indicate by $\omega_n \cdot \sin(\theta) = 0.9425i$.

So the desired pole location of our balancing robot is **$p_1 = -3.7699$, $p_2 = -3.7699 + 0.9425i$, and $p_3 = -3.7699 - 0.9425i$** .

Once the desired pole locations are determined, we put them into the MATLAB script Rocky_closed_loop_poles_24. It calculates the controller gains k_p and k_i . These gain values are then implemented in the Arduino balance starter code, which sends control commands to the balancing robot.

However, the robot does not balance well with this controller alone as what we expected. Since the system does not yet include a velocity controller, the robot tends to accelerate quickly and eventually falls after running too fast.

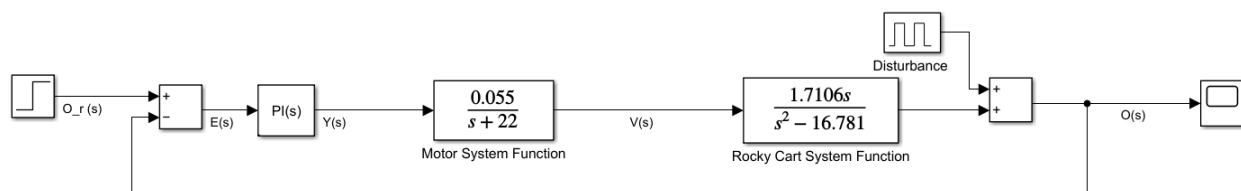


Figure 4 : System Diagram for the 3-pole system in Simulink

The simulink model represents the three-pole closed-loop control system used to stabilize the balancing robot. The controller regulates the robot's angle by comparing the measured angle with the reference (0°) and sending a control signal to the motor system.

Three-Pole System Figure

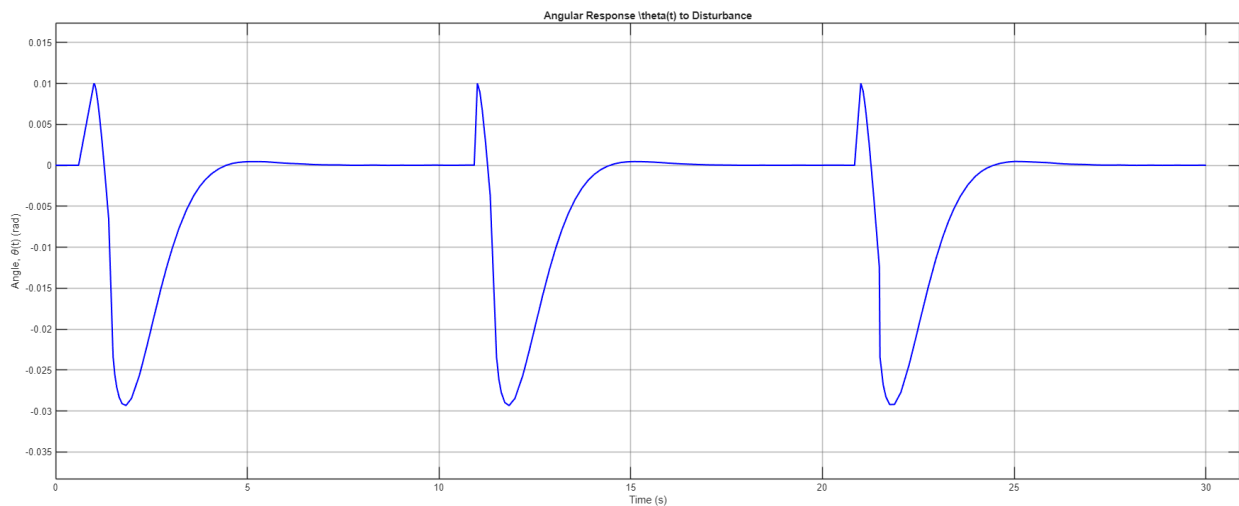


Figure 5: Response of Rocky's angular position to a disturbance for the initial three-pole balancing system. After each disturbance, the angle deviates from zero and then settles back near zero, showing that the controller is able to balance Rocky upright..

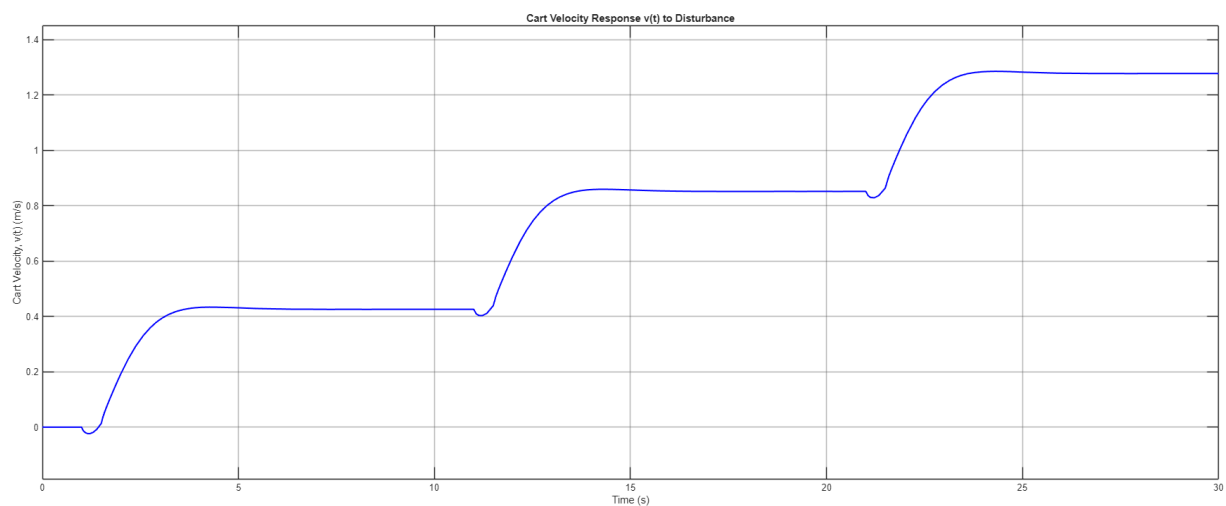


Figure 6: Response of Rocky's linear velocity to a disturbance for the initial three-pole balancing system. After each disturbance, the cart velocity changes as Rocky moves to recover balance, but it does not regulate back to zero, showing that this controller does not enforce stationary balancing.

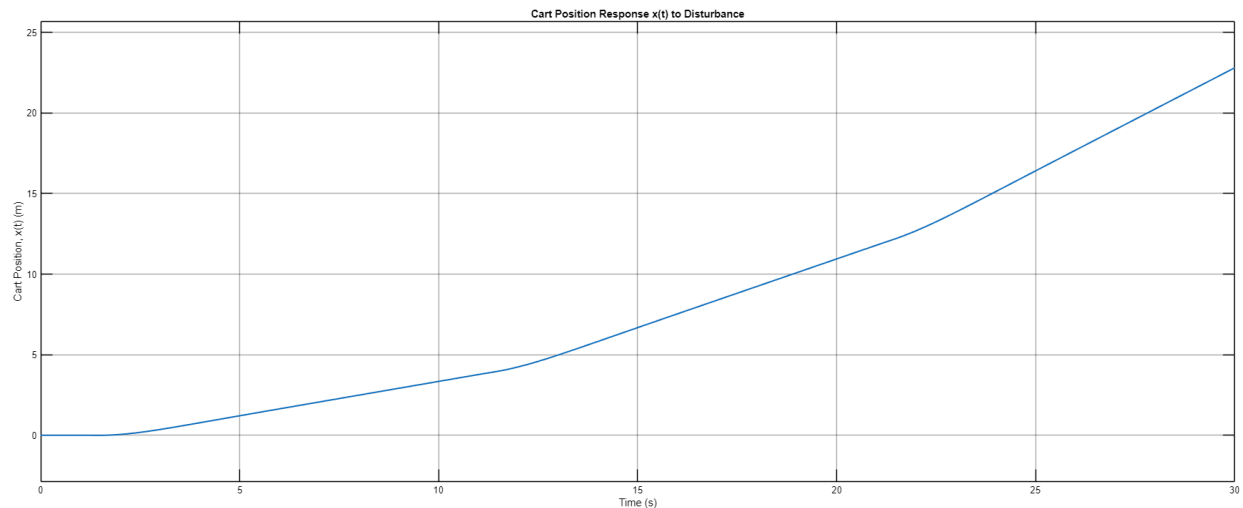


Figure 7: Response of Rocky's displacement to a disturbance for the initial three-pole balancing system. The displacement continues to drift and does not return to zero, showing that the controller balances Rocky but does not bring it back to its original position..

The response plot shows the disturbances happen in approximately 0s, 10s, and 20s, and it causes sudden deviation in the angle displacement. The system recovers and brings the angle back to 0 to make sure the robot is balanced. However, as I mentioned before, the real situation is not as good as the prediction. The controller only stabilizes and regulates the angle, but there is no control of the velocity of the wheels. That causes the balancing robot to lose balance because the wheel accelerates too quickly while it tries to maintain the angle value.

Stationary Balancing System

To achieve stationary balancing, we augmented the initial disturbance rejection balancing controller with an additional cart feedback structure so the robot not only returns its body angle to $\theta \approx 0$ but also returns its cart position back toward the starting location after a disturbance. The plant relating cart velocity to body angle is modeled as

$$H_{v\theta}(s) = \frac{\Theta(s)}{V(s)} = \frac{-s/l_{\text{eff}}}{s^2 - g/l_{\text{eff}}}$$

The motor dynamics are modeled as a first-order transfer function:

$$M(s) = \frac{V(s)}{Y(s)} = \frac{ab}{s + a}$$

where a and b are the identified motor constants. The inner controller is PI:

$$K(s) = K_p + \frac{K_i}{s}$$

To prevent drift and return the cart to its original position after a disturbance, we introduce a feedback controller around the motor subsystem:

$$J(s) = J_p + \frac{J_i}{s} + \frac{C_i}{s^2}$$

Using Black's formula, the effective motor transfer function with feedback becomes:

$$M_{fb}(s) = \frac{M(s)}{1 + M(s)J(s)}$$

With the disturbance injected at the angle output, the stationary closed-loop disturbance to angle transfer function is:

$$H_{cl}(s) = \frac{\Theta(s)}{D(s)} = \frac{1}{1 - H_{v\theta}(s) M_{fb}(s) K(s)}$$

For our calculations, we used $g=9.81 \text{ m/s}^2$, $l_{\text{eff}}=0.5840 \text{ m}$, and nominal motor values $a=22$, $b=0.0025$.

Stationary Pole Locations and Rationale

Because the stationary controller includes motor dynamics and additional feedback around cart motion, the closed-loop system is modeled as a higher-order system. We selected the target closed-loop poles to produce a fast, well-damped recovery in body angle while avoiding overly aggressive cart motion that could lead to motor saturation. The selected target poles were:

$$p_{1,2}=-3\pm 2i, p_3=-3, p_4=-4, p_5=-2.$$

The complex conjugate pair $(-3\pm 2i)$ shapes the oscillatory part of the response, while the real pole at -2 contributes a slower non-oscillatory mode that affects the final settling behavior. For this pair, the decay rate is $\sigma=3 \text{ s}^{-1}$ and the damped oscillation frequency is $\omega_d=2 \text{ rad/s}$. The remaining real poles shape the non-oscillatory modes associated with cart/motor feedback and were chosen to keep the response stable while still allowing the cart to return toward $x=0$ after disturbances.

Calculation of Stationary Control Constants

We computed controller gains by matching the coefficients of the closed-loop characteristic polynomial to the polynomial defined by the target pole locations. Using the symbolic form of $H_{cl}(s)$ and substituting system parameters $g=9.81 \text{ m/s}^2$, $I_{eff}=0.5840 \text{ m}$, $a=22$, and $b=0.0025$, we extracted the closed-loop denominator and solved for the controller parameters (K_p, K_i, J_p, J_i, C_i) via coefficient matching in MATLAB. The resulting gains were:

$$K_p=1.4705\times 10^3, K_i=6.0263\times 10^3, J_p=-127.2727, J_i=-521.7088, C_i=-337.7036.$$

These gains were then implemented in the stationary balancing controller structure.

Stationary Response to Disturbance (Closed-Loop Behavior)

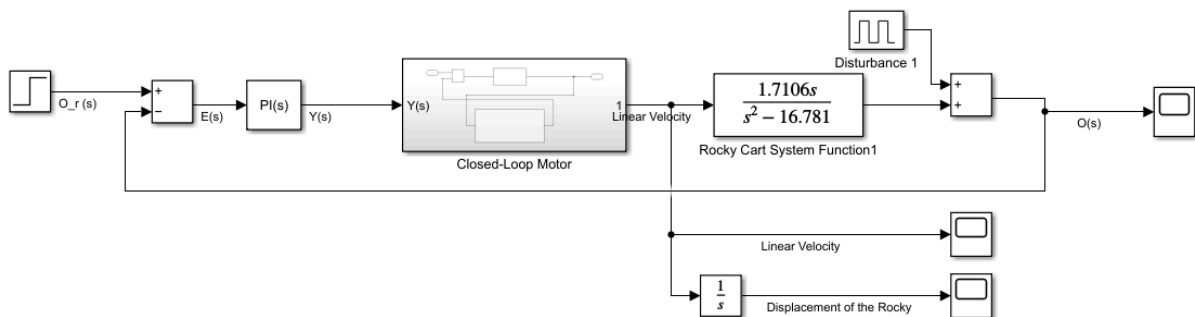


Figure 8 : System Diagram for the 5-pole system in Simulink

The Simulink model represents the five-pole closed loop control system used to stabilize the balancing robot. Compare with 3 pole control system, This system improves stability and better regulation of both the robot angle and the wheel dynamics by adding additional poles to the controller design.

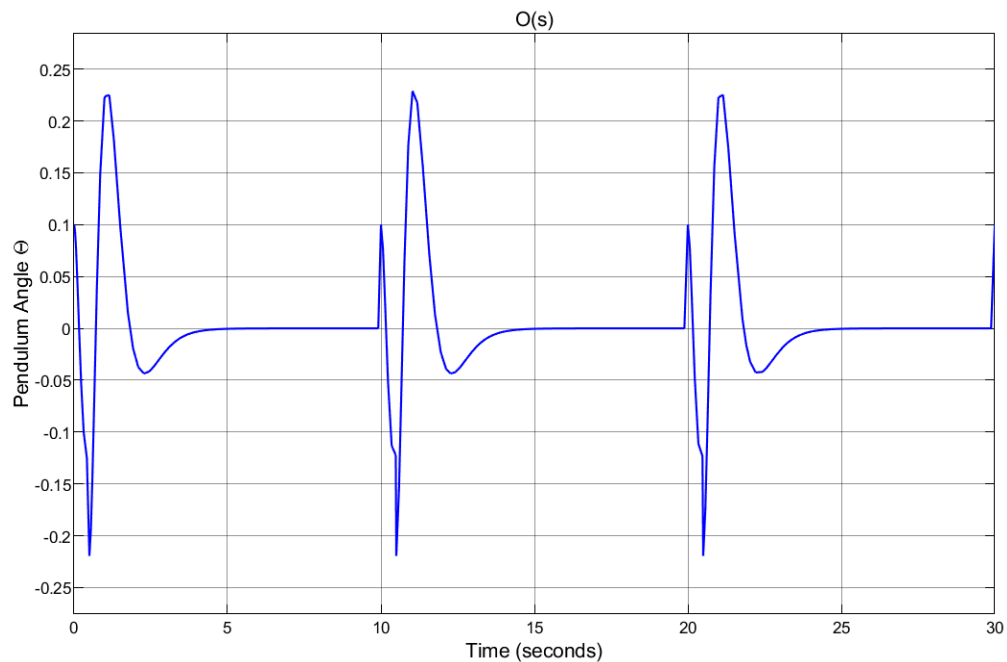


Figure 9 : Response of Rocky's angular position to a disturbance for the five-pole stationary balancing system. After each disturbance, the angle briefly deviates from zero and then settles back, showing disturbance rejection and stable balancing.

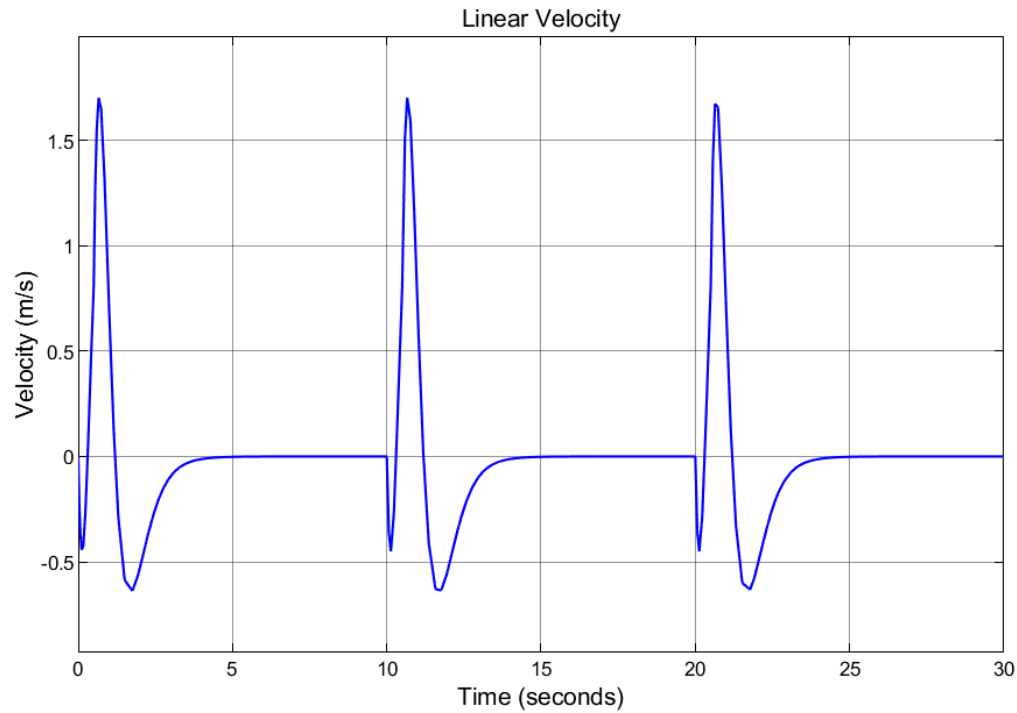


Figure 10 : Response of Rocky's linear velocity to a disturbance for the five-pole stationary balancing system. After each disturbance, the velocity briefly changes and then returns to zero, indicating that the controller regulates Rocky's motion..

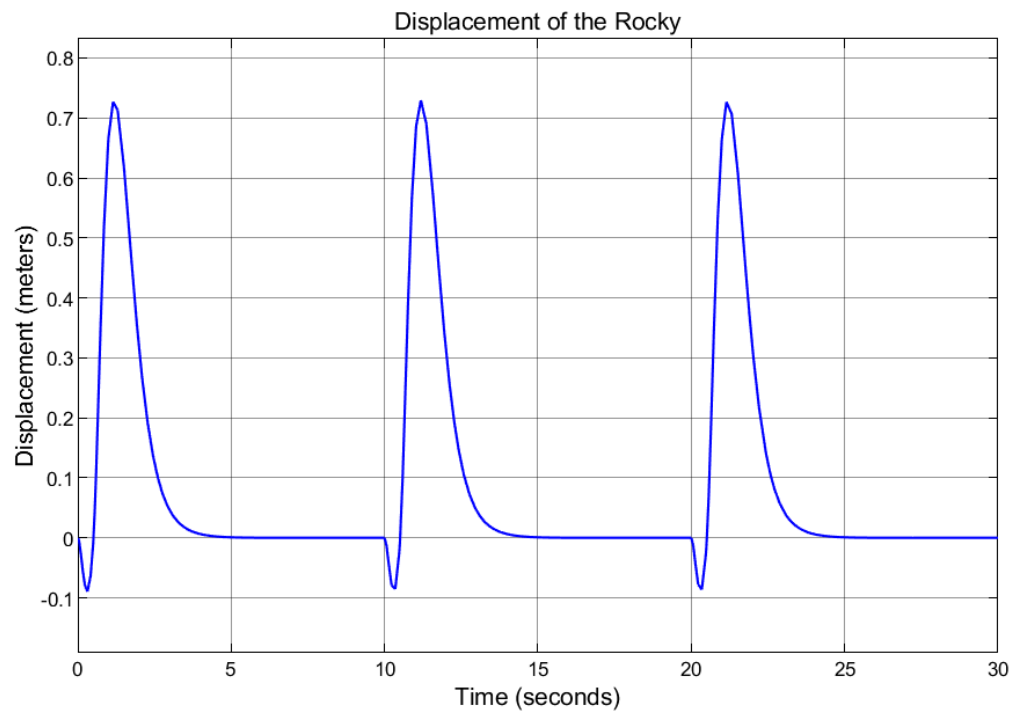


Figure 11 : Response of Rocky's displacement to a disturbance for the five-pole stationary balancing system. After each disturbance, the displacement returns close to zero, showing that the controller reduces drift and brings Rocky back toward its original position..

All three diagrams demonstrate that when encountering disturbances, the control system adjusts to return the angle, velocity, and displacement to zero, consistent with the actual robot's performance as shown in the video show.

Simulink vs Rocky comparison

The five-pole stationary balancing controller performed better than the initial three-pole controller because it not only stabilized the angle of Rocky, but also returned the cart velocity and displacement toward zero after each disturbance. In the Simulink model, all three responses settled after each pulse disturbance, showing both balancing and position regulation. The physical Rocky showed similar overall behavior, but with more noise and slightly less smooth settling than the simulation due to real-world effects such as friction, motor mismatch, and sensor noise.

[Video](https://youtube.com/shorts/4PalO6uJMQc?feature=share) (<https://youtube.com/shorts/4PalO6uJMQc?feature=share>)

Code

Finding Poles of 3 Pole Transfer Function

```
clear; clf;

zeta_pole = 0.8;

wn = pi/2;

theta_pole = acosd(zeta_pole);

a_pole = -3.*wn.*cosd(theta_pole);

q1 = wn.*sind(theta_pole)*i;

q2 = - q1;
```

```
p1 = a_pole + q1
p2 = a_pole + q2
p3 = a_pole
```

```

figure(); hold on
plot(p1, '*');
plot(p2, '*');
plot(p3, 0, '*');

title('Poles of our Closed (3 Pole) Transfer Function');
xlabel('Real'); ylabel('Imaginary')
legend('Pole 1', 'Pole2', 'Pole 3')

```

Finding Ki and Kp values

```

close all;

syms s a b l g Kp Ki Jp Ji Ci % define symbolic variables
Hvtheta = (-s/l)/(s^2-g/l); % TF from velocity to angle of pendulum
K = Kp + Ki/s; % TF of the PI angle controller
M = (a*b)/(s+a); % TF of motor (1st order model)

```

```

% closed loop transfer function from disturbance d(t) to theta(t)
Hcloop = 1/(1-Hvtheta*M*K); % use this for no motor feedback
pretty(simplify(Hcloop)) % to display the total transfer function

```

```

% Substitute parameters and solve
% system parameters
g = 9.81;
l = 0.5840; %effective length
a = 22; %nominal motor parameters
b = 0.0025; %nominal motor parameters
Hcloop_sub = subs(Hcloop)
% sub parameter values into Hcloop

```

```

% specify locations of the target poles,

```

```

tgt_char_poly = (s-p1)*(s-p2)*(s-p3);
npoly = 3;

% get the denominator from Hcloop_sub

[n d] = numden(Hcloop_sub);

```

```

% find the coefficients of the denominator polynomial TF
coeffs_denom = coeffs(d, s);

% divide though the coefficient of the highest power term
coeffs_denom = coeffs(d, s)/(coeffs_denom(end));

% find coefficients of the target charecteristic polynomial
coeffs_tgt = coeffs(tgt_char_poly, s);

% solve the system of equations setting the coefficients of the
% polynomial in the target to the actual polynomials
solutions = solve(coeffs_denom(1:npoly-1) == coeffs_tgt(1:npoly-1), Kp,
Ki);

% display the solutions as double precision numbers
Kp = double(solutions.Kp)
Ki = double(solutions.Ki)

% reorder coefficients for the check polynomial
for ii = 1:length(coeffs_denom)
chk_coeffs_denom(ii) = coeffs_denom(length(coeffs_denom) + 1 - ii);
end

closed_loop_poles = vpa (roots(subs(chk_coeffs_denom)), npoly )

% Plot impulse response of closed-loop system
TFstring = char(subs(Hcloop));

% Define 's' as transfer function variable
s = tf('s');

% Evaluate the expression

```

```

eval(['TFH = ',TFstring]);

figure (1)

impz(TFH); %plot the impulse reponse

figure(2)

step(TFH) %plot the step response

```

Finding Jp, Ji, Ci

```

clear all;

close all;

syms s a b l g Kp Ki Jp Ji Ci % define symbolic variables

Hvtheta = -s/l/(s^2-g/l); % TF from velocity to angle of pendulum

K = Kp + Ki/s; % TF of the PI angle controller

M = a*b/(s+a); % TF of motor (1st order model)

% M = 1; % TF without motor

%

J = Jp + Ji/s + Ci/s^2; % TF of controller around motor-combined PI of x

Mfb = M/(1+M*J); % Black's formula to get tf for motor with PI feedback
control

%

% closed loop transfer function from disturbance d(t) to theta(t)

% Hcloop = 1/(1-Hvtheta*M*K) % use this for no motor feedback

% with motor feedback

Hcloop = 1/(1-Hvtheta*Mfb*K) % use this for motor with feedback

pretty(simplify(Hcloop)) % to display the total transfer function

% Substitute parameters and solve

% system parameters

g = 9.81;

l = 0.5840; %effective length

```

```

a = 22; %nominal motor parameters
b = 0.0025; %nominal motor parameters
Hcloop_sub = subs(Hcloop) % sub parameter values into Hcloop
% specify locations of the target poles,
% choose # based on order of Htot denominator
% e.g., want some oscillations, want fast decay, etc.

```

```

p1 = -3 + 2*i % dominant pole pair
p2 = -3 -2*i % dominant pole pair
p3 = -3
p4 = -4 % dominant pole pair
p5 = -2 % dominant pole pair
% target characteristic polynomial
% if motor model (TF) is added, order of polynomial will increase
% tgt_char_poly = (s-p1)*(s-p2)*(s-p3)
% check polynomial-expand to fifth order
tgt_char_poly = (s-p1)*(s-p2)*(s-p3)*(s-p4)*(s-p5);
exp_tgt_char_poly = expand(tgt_char_poly);
% get the denominator from Hcloop_sub
[n d] = numden(Hcloop_sub);
% find the coefficients of the denominator polynomial TF
coeffs_denom = coeffs(d, s);
% divide through the coefficient of the highest power term
coeffs_denom = coeffs(d, s)/(coeffs_denom(end));
% num_coeff_denom = length(coeffs_denom)
% find coefficients of the target characteristic polynomial
coeffs_tgt = coeffs(tgt_char_poly, s);
% num_coeff_tgt = length(coeffs_tgt)

```

```

% for check. reorder the coefficients to match the denominator
polynomial

for ii = 1:length(coeffs_denom)
reord_coeffs_tgt(ii) = coeffs_tgt(length(coeffs_tgt) + 1 - ii);
end

% check roots of target polynomial-should be same as selected poles
roots_target = vpa(roots(reord_coeffs_tgt),4);

% solve the system of equations setting the coefficients of the
% polynomial in the target to the actual polynomials
solutions = solve(coeffs_denom(1:5) == coeffs_tgt(1:5), Jp, Ji, Kp, Ki,
Ci);

% display the solutions as double precision numbers
Kp = double(solutions.Kp)
Ki = double(solutions.Ki)
Ji = double(solutions.Ji)
Jp = double(solutions.Jp)
Ci = double(solutions.Ci)

%write out denominator polynomial
aaa = vpa(subs(coeffs_denom),4)

% reorder coefficients for the check polynomial
for ii = 1:length(coeffs_denom)
chk_coeffs_denom(ii) = coeffs_denom(length(coeffs_denom) + 1 - ii);
end

% check poles should be same as chosen input poles
check_closed_loop_poles = vpa (roots(subs(chk_coeffs_denom)), 4);

% write out target polynomial
% bbb = vpa( expand( (s-check_closed_loop_poles(1))*(s-
% *(s-check_closed_loop_poles(3))*(s-check_closed_loop_poles(4)) ...

```

```

% *(s-check_closed_loop_poles(5)) ) )

% Plot impulse and step responses of closed-loop system

TFstring = char(subs(Hcloop));

% Define 's' as transfer function variable

s = tf('s');

% Evaluate the expression

eval(['TFH = ',TFstring]);

figure (1)

impz(TFH); %plot the impulse reponse

figure(2)

step(TFH) %plot the step response

```

Calculating the Motor Parameters from Data

```

motor_data = load('Rocky_Motor_Data.txt');

t = motor_data(:,1);

Y_L = 300;

Y_R = 300;

v_L = motor_data(:,4);

v_R = motor_data(:,5);

%Estimate the parameters for the right wheel

t = t(101:200)-2;

v_R = v_R(101:200);

%Fit a model to the motor data

%Initial Guesses

a_g = 0.7; c_g = 2;

```

```
% Linear fit for right wheel velocity
V_params = fit(t, v_R, 'c*(1-exp(-a*x))', 'StartPoint', [a_g, c_g]);
a = V_params.a; c = V_params.c;

%Evaluate the data
v_R_fit = c*(1-exp(-a*t));
```

```
%compare fit with the data
figure; hold on
plot(t,v_R,'ro','markerfacecolor','r','markersize',3);
plot(t,v_R_fit,'k','linewidth',2);
xlabel('Time (sec)'); ylabel('V_R(t)');
title("Custom Fit Model on the Right Wheel's Linear Velocity");
legend('Data','Fit Model', 'location','southeast');
```

```
% Compute  $b = c/Y$  and  $\tau = 1/a$  from the fit
b = c ./ Y_R
tau = 1 ./ a
```

Calculating the Natural Frequency Using Data

```
% Load the Rocky Data
P = load('Rocky_Angle_Data.txt');

%unpack the experimental data
t_p = P(:,1)-1; %time (sec)
theta = P(:,2); %pendulum angle (rad)
```

```
% generate a fit to the response data
pendulum_params = regress_underdamped_response(t_p, theta);
```

```

% Unpack the data

C = pendulum_params.C;

S = pendulum_params.S;

D = pendulum_params.D;

w_n = pendulum_params.omega_n;

zeta = pendulum_params.zeta;

w_d = pendulum_params.omega_d;

sigma = pendulum_params.sigma;


% Create the function

theta_fit = D + (C.*cos(w_d.*t_p) + S.*sin(w_d.*t_p)).*exp(-sigma.*t_p);


% Plot the Measured vs Predicted data

figure(); hold on

plot(t_p, theta, 'k', 'LineWidth',2)

plot(t_p, theta_fit, 'r--', 'LineWidth',2)

title('Rocky Pendulum Free Response Data');

xlabel('t (sec)'); ylabel('angle (rad)');

legend('Measured Data', 'Fit Data');


% Use the natural Frequency to calculate Length

fprintf('The natural frequency of the system is %.3f rad/sec', w_n);

```

Parameters for the Simulink Model

```

D_s = 1; % Disturbance Force magnitude (square wave)

K_p = Kp; % Proportional controller Constant

K_i = Ki; % Integral Controller Constant

alpha = a; % Motor Constant = 1/torque

```

```

beta = b; % Motor Constant = K*alpha

g = 9.81; %Gravity m/s^2

l_eff = abs(g/(w_n.^2)) %length of the rocky in meters

```

Arduino

```

// Start the robot flat on the ground
// compile and load the code
// wait for code to load (look for "done uploading" in the Arduino IDE)
// wait for red LED to flash on board
// gently lift body of rocky to upright position
// this will enable the balancing algorithm
// wait for the buzzer
// let go
//
// The balancing algorithm is implemented in BalanceRocky()
// which you should modify to get the balancing to work
//

#include <Balboa32U4.h>
#include <Wire.h>
#include <LSM6.h>
#include "Balance.h"

extern int32_t angle_accum;
extern int32_t speedLeft;
extern int32_t driveLeft;
extern int32_t distanceRight;
extern int32_t speedRight;
extern int32_t distanceLeft;
extern int32_t distanceRight;
float speedCont = 0;
float displacement_m = 0;
int16_t limitCount = 0;
uint32_t cur_time = 0;
float distLeft_m;
float distRight_m;

```

```

extern uint32_t delta_ms;
float measured_speedL = 0;
float measured_speedR = 0;
float desSpeedL=0;
float desSpeedR =0;
float dist_accumL_m = 0;
float dist_accumR_m = 0;
float dist_accum = 0;
float speed_err_left = 0;
float speed_err_right = 0;
float speed_err_left_acc = 0;
float speed_err_right_acc = 0;
float errAccumRight_m = 0;
float errAccumLeft_m = 0;
float prevDistLeft_m = 0;
float prevDistRight_m = 0;
float angle_rad_diff = 0;
float angle_rad;           // this is the angle in radians
float angle_rad_accum = 0; // this is the accumulated angle in
radians
float angle_prev_rad = 0; // previous angle measurement
extern int32_t displacement;
int32_t prev_displacement=0;
uint32_t prev_time;

#define G_RATIO (162.5)

LSM6 imu;
Balboa32U4Motors motors;
Balboa32U4Encoders encoders;
Balboa32U4Buzzer buzzer;
Balboa32U4ButtonA buttonA;

#define FIXED_ANGLE_CORRECTION (0.26) // ***** Replace the value 0.25
with the value you obtained from the Gyro calibration procedure

```

```

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////
// This is the main function that performs the balancing
// It gets called approximately once every 10 ms by the code in loop()
// You should make modifications to this function to perform your
// balancing
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////

void BalanceRocky()
{

    // *****Enter the control parameters here

    float Kp = 1.4705e+03;
    float Ki = 6.0263e+03;
    float Ci = -337.7036;
    float Jp = -127.2727;
    float Ji = -521.7088;

    float v_c_L, v_c_R; // these are the control velocities to be sent to
the motors
    float v_d = 0; // this is the desired speed produced by the angle
controller

    // Variables available to you are:
    // angle_rad - angle in radians
    // angle_rad_accum - integral of angle
    // measured_speedR - right wheel speed (m/s)
    // measured_speedL - left wheel speed (m/s)
    // distLeft_m - distance traveled by left wheel in meters

```

```

    // distRight_m - distance traveled by right wheel in meters (this is
the integral of the velocities)
    // dist_accum - integral of the distance

    // *** enter an equation for v_d in terms of the variables available
****
    v_d = (Kp * angle_rad + Ki * angle_rad_accum);

    // The next two lines implement the feedback controller for the motor.
Two separate velocities are calculated.
    //
    //
    // We use a trick here by criss-crossing the distance from left to right
and
    // right to left. This helps ensure that the Left and Right motors are
balanced

    // *** enter equations for input signals for v_c (left and right) in
terms of the variables available ****
    v_c_R = v_d - (Jp * measured_speedR + Ji * distLeft_m + Ci *
dist_accum);
    v_c_L = v_d - (Jp * measured_speedL + Ji * distRight_m + Ci *
dist_accum);

    // save desired speed for debugging
    desSpeedL = v_c_L;
    desSpeedR = v_c_R;

    // the motor control signal has to be between +- 300. So clip the
values to be within that range
    // here
    if(v_c_L > 300) v_c_L = 300;
    if(v_c_R > 300) v_c_R = 300;
    if(v_c_L < -300) v_c_L = -300;

```

```

    if(v_c_R < -300) v_c_R = -300;

    // Set the motor speeds
    motors.setSpeeds((int16_t) (v_c_L), (int16_t) (v_c_R));

}

```

```

void setup()
{
    // Uncomment these lines if your motors are reversed.
    // motors.flipLeftMotor(true);
    // motors.flipRightMotor(true);

    Serial.begin(9600);
    prev_time = 0;
    displacement = 0;
    ledYellow(0);
    ledRed(1);
    balanceSetup();
    ledRed(0);
    angle_accum = 0;

    ledGreen(0);
    ledYellow(0);
}

```

```

int16_t time_count = 0;
extern int16_t angle_prev;
int16_t start_flag = 0;
int16_t start_counter = 0;
void lyingDown();
extern bool isBalancingStatus;
extern bool balanceUpdateDelayedStatus;

void UpdateSensors()
{

```

```

static uint16_t lastMillis;
uint16_t ms = millis();

// Perform the balance updates at 100 Hz.
balanceUpdateDelayedStatus = ms - lastMillis > UPDATE_TIME_MS + 1;
lastMillis = ms;

// call functions to integrate encoders and gyros
balanceUpdateSensors();

if (imu.a.x < 0)
{
    lyingDown();
    isBalancingStatus = false;
}
else
{
    isBalancingStatus = true;
}
}

void GetMotorAndAngleMeasurements()
{
    // convert distance calculation into meters
    // and integrate distance
    distLeft_m =
((float)distanceLeft)/((float)G_RATIO)/12.0*80.0/1000.0*3.14159;
    distRight_m =
((float)distanceRight)/((float)G_RATIO)/12.0*80.0/1000.0*3.14159;
    dist_accum += (distLeft_m+distRight_m)*0.01/2.0;

    // compute left and right wheel speed in meters/s
    measured_speedL =
speedLeft/((float)G_RATIO)/12.0*80.0/1000.0*3.14159*100.0;
    measured_speedR =
speedRight/((float)G_RATIO)/12.0*80.0/1000.0*3.14159*100.0;

    prevDistLeft_m = distLeft_m;

```

```

prevDistRight_m = distRight_m;

// this integrates the angle
angle_rad_accum += angle_rad*0.01;
// this is the derivative of the angle
angle_rad_diff = (angle_rad-angle_prev_rad)/0.01;
angle_prev_rad = angle_rad;
}

void balanceResetAccumulators()
{
    errAccumLeft_m = 0.0;
    errAccumRight_m = 0.0;
    speed_err_left_acc = 0.0;
    speed_err_right_acc = 0.0;
}

void loop()
{
    static uint32_t prev_print_time = 0;    // this variable is to control
how often we print on the serial monitor
    int16_t distanceDiff;    // this stores the difference in distance in
encoder clicks that was traversed by the right vs the left wheel
    static float del_theta = 0;
    char enableLongTermGyroCorrection = 1;

    cur_time = millis();    // get the current time in
milliseconds

    if((cur_time - prev_time) > UPDATE_TIME_MS){
        UpdateSensors();    // run the sensor updates.

        // calculate the angle in radians. The FIXED_ANGLE_CORRECTION term
comes from the angle calibration procedure (separate sketch available for
this)

```

```

    // del_theta corrects for long-term drift
    angle_rad = ((float)angle)/1000/180*3.14159 - FIXED_ANGLE_CORRECTION -
del_theta;

    if(angle_rad > 0.1 || angle_rad < -0.1)        // If angle is not within
+- 6 degrees, reset counter that waits for start
    {
        start_counter = 0;
    }

    if(angle_rad > -0.1 && angle_rad < 0.1 && ! start_flag)
    {
        // increment the start counter
        start_counter++;
        // If the start counter is greater than 30, this means that the angle
has been within +- 6 degrees for 0.3 seconds, then set the start_flag
        if(start_counter > 30)
        {
            balanceResetEncoders();
            start_flag = 1;
            buzzer.playFrequency(DIV_BY_10 | 445, 1000, 15);
            Serial.println("Starting");
            ledYellow(1);
        }
    }

    // every UPDATE_TIME_MS, if the start_flag has been set, do the
balancing
    if(start_flag)
    {
        GetMotorAndAngleMeasurements();
        if(enableLongTermGyroCorrection)
            del_theta = 0.999*del_theta + 0.001*angle_rad; // assume that the
robot is standing. Smooth out the angle to correct for long-term gyro
drift

        // Control the robot
        BalanceRocky();
    }

```

```

    }
    prev_time = cur_time;
}
// if the robot is more than 45 degrees, shut down the motor
if(start_flag && angle_rad > .78)
{
    motors.setSpeeds(0,0);
    start_flag = 0;
}
else if(start_flag && angle < -0.78)
{
    motors.setSpeeds(0,0);
    start_flag = 0;
}

// kill switch
if(buttonA.getSingleDebouncePress())
{
    motors.setSpeeds(0,0);
    while(!buttonA.getSingleDebouncePress());
}

if(cur_time - prev_print_time > 103)    // do the printing every 105 ms.
Don't want to do it for an integer multiple of 10ms to not hog the
processor
{
    Serial.print(angle_rad);
    Serial.print("\t");
    Serial.print(distLeft_m);
    Serial.print("\t");
    Serial.print(measured_speedL);
    Serial.print("\t");
    Serial.print(measured_speedR);
    Serial.print("\t");
    Serial.println(speedCont);
    prev_print_time = cur_time;
}
}

```

